PART 1

1) Row and key Sharding

I would recommend using the VIN for the row key with the prefix hash. Using the hash distributes the records instead of putting everything in one place. Using this method would prevent vehicle nodes from hotspotting and keep the workload at an equilibrium. When it comes to searching by City or Make, however, the process would take longer due to the values not starting at the row key. If we would want to make the searches run faster, we would need to make separate lookup tables like City#VIN.

2) Column Families

The way I would design the column families for this dataset is but putting them into 3 categories:

- vehicles: VIN, Model, Make, Model Year, and Electric Vehicle Type
- location: City, State, Country, Postal Code, and Vehicle Location
- Ev_specs: Base, Electric Range, Base MSRP, and CAFV Eligibility

The reason why grouping is more efficient is due to how easy it is to lookup and receive data versus when it's not categorized. For example, a query searching for vehicle locations would need the location family instead of going through and reading every attribute.

3) Horizontal Scaling

Hashed Shard Key (VIN)

- Pros: Distributes data across nodes which balances everything out and prevents any hotspotting
- Cons: Searching by ranges or by certain tags will become more difficult due to the data/records not maintaining their original order.

Range-Based Shard Key

- Pros: More efficient when searching for vehicles from a specific range of years or a year in general.
- Cons: Hotspotting can occur due to a large scale of records being under one year range.

PART 4

Average Latency

Endpoint	Average latency
/insert-fast	29.746 ms
/insert-safe	31.678 ms

CAP Theorem Analysis:

The fast write (w=1) is a lot faster due to it waiting for confirmations from the primary node, but the caveat is that recent written data could be lost if the primary falls without prior replication. A good example would be the traffic monitoring system. Losing a few sensor readings would be okay so I would choose the faster process instead.